

HashiCorp Vault, Consul & Chef Automate

Licensing, Recovery, and LDAP Troubleshooting — A practical guide to managing enterprise infrastructure tools with confidence.



Agenda: What We'll Cover Today

01

License Management

Updating Vault and Consul licenses in online and air-gapped environments

02

Vault Restart Recovery

Diagnosing and fixing the five most common reasons Vault fails to restart

03

Chef Automate LDAP

Resolving authentication failures when LDAP users appear in policy but still can't sign in



Why License Management Matters

Service Disruption

An expired license can cause Vault or Consul to enter a degraded or read-only state, blocking secret access across all dependent services.

Compliance Risk

Running unlicensed enterprise features violates BSL terms and can trigger audit findings — especially in regulated industries.

Silent Failures

Misconfigured licenses may not immediately surface as errors, creating a false sense of security until a critical moment.

HashiCorp's Licensing Model



What Changed in 2023

HashiCorp moved from the Mozilla Public License (MPL) to the **Business Source License (BSL 1.1)** in August 2023. This affects Vault, Consul, Terraform, and others.

BSL (Enterprise)

Production use requires a paid license. Feature-rich with official support.

Open Fork (OpenTF)

OpenTofu and other community forks remain under MPL for open-source users.

STEP 1

Checking Your Current Vault License

CLI Method

Use the Vault CLI to retrieve license details directly:

```
$ vault license get
```

This returns the license **expiration date**, licensed features, and the **license ID**. Ensure `VAULT_ADDR` and `VAULT_TOKEN` environment variables are set before running.

UI Method

Navigate to **Admin** → **License** in the Vault web UI. The dashboard displays the license status, expiry countdown, and any feature restrictions.

📌 **Pro Tip:** Set up a monitoring alert when license expiry is within 30 days. Use `vault license inspect` to parse the raw license blob without applying it.

STEP 2

Applying a New License to HashiCorp Vault

Online Environment

Apply via the CLI in a connected environment:

```
$ vault write sys/license \
  text="<LICENSE_STRING>"
```

The license takes effect immediately without a restart.

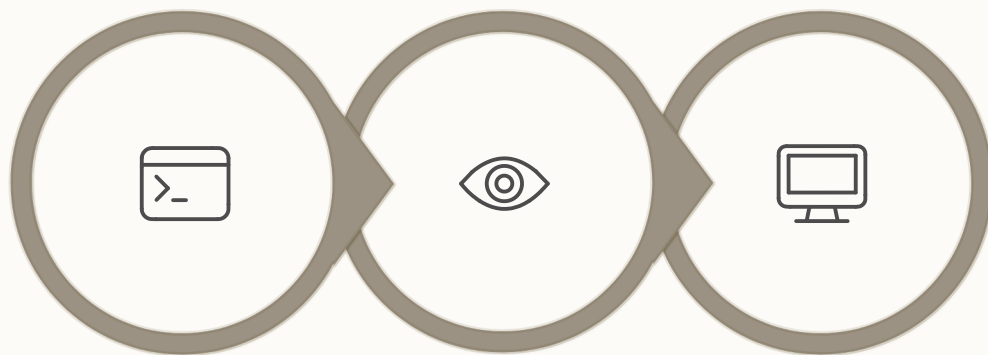
Air-Gapped Environment

Set the license as an environment variable before starting Vault:

```
VAULT_LICENSE="<LICENSE_STRING>"
```

Or place it in the Vault config file using the `license_path` directive pointing to a local file.

Verifying the Vault License Was Applied



Run
Command

Check Output

Validate UI

What to Look For

- **Expiration date** should reflect the new term
- **Licensed features** should include all expected enterprise modules (e.g., HSM, Replication, MFA)
- The `license_id` field should match the ID provided by HashiCorp support

❑ If the old license still appears after applying, flush the cache and re-authenticate your token with appropriate permissions.



Config File vs. API — Which Takes Precedence?

License in Config File

Defined via `license_path` in `vault.hcl`. Loaded at startup. Can be overridden at runtime by an API call. Useful for air-gapped or immutable infrastructure deployments.

License via API

Applied using `vault write sys/license`. Takes effect immediately and **persists in storage**. This takes precedence over the config file in most versions.

Best Practice

Keep both in sync. After applying via API, update your config file and secrets management pipeline to avoid drift during the next restart cycle.

Changing the Consul License

Using the CLI

```
$ consul license put \  
  @/path/to/license.hcl
```

This applies the license to the leader node. It will **replicate automatically** to all followers in the cluster.

Using an Environment Variable

```
CONSUL_LICENSE="<LICENSE_STRING>"
```

Set this before starting the Consul agent. This is the preferred approach for containerized or systemd-managed deployments.

Using the HTTP API

```
PUT /v1/operator/license  
Content-Type: application/json  
{  
  "License": "<LICENSE_STRING>"  
}
```

- ❏ Always use `-token` or set `CONSUL_HTTP_TOKEN` when ACLs are enabled, or the request will be rejected with a 403.

Validating the Consul License Across All Nodes

1 Check the Leader

Run `consul license get` on the leader node first. Confirm the expiry date and licensed feature set match expectations.

2 Validate Followers

SSH into each follower and run `consul license get`. Replication should have propagated the license automatically within seconds.

3 Check Consul UI

In the Consul UI, navigate to **Settings** → **License**. Review the cluster-wide license status and any warnings about upcoming expiry.

Vault vs. Consul License Updates: Key Differences

Vault

- License applied via `sys/license` API or `license_path` config
- Stored in Vault's backend storage
- No restart required for API-applied licenses
- Each cluster member reads from shared storage



Consul

- License applied via `consul license put` or env var
- Replicates from leader to all followers automatically
- Env var method requires agent restart
- Must validate on every node individually

Why Does Vault Fail to Restart?

The five most common root causes engineers encounter in production environments:



Seal Config Error

Auto-unseal KMS unreachable or misconfigured



TLS Failure

Expired or missing certificates
block listener startup



Storage Backend

Consul, Raft, or etcd unavailable or misconfigured



Env Var Conflicts

Overriding config values causing unexpected behavior



Permission Errors

Wrong file ownership or systemd service account

Diagnosing Vault Restart Failures

Reading journalctl Logs

```
$ journalctl -u vault.service \
-n 200 --no-pager
```

Look for `ERROR` or `FATAL` lines. Common patterns include **"core: vault is sealed"**, **"listener config error"**, or **"backend initialization failed"**.

Vault Audit Log

```
$ tail -f /var/log/vault/audit.log \
| jq .
```

If Vault didn't start, the audit log may be empty — check the system log first. Enable `-log-level=debug` for verbose output.

📌 **Quick Tip:** Add `VAULT_LOG_LEVEL=debug` to your systemd unit file temporarily. This surfaces configuration parsing errors that are otherwise suppressed at the default log level.

Fixing Seal Configuration Errors



AWS KMS Auto-Unseal

Verify the IAM role attached to the Vault EC2 instance has `kms:Decrypt` and `kms:DescribeKey` permissions. Check `region` and `kms_key_id` in `vault.hcl`.



Azure Key Vault

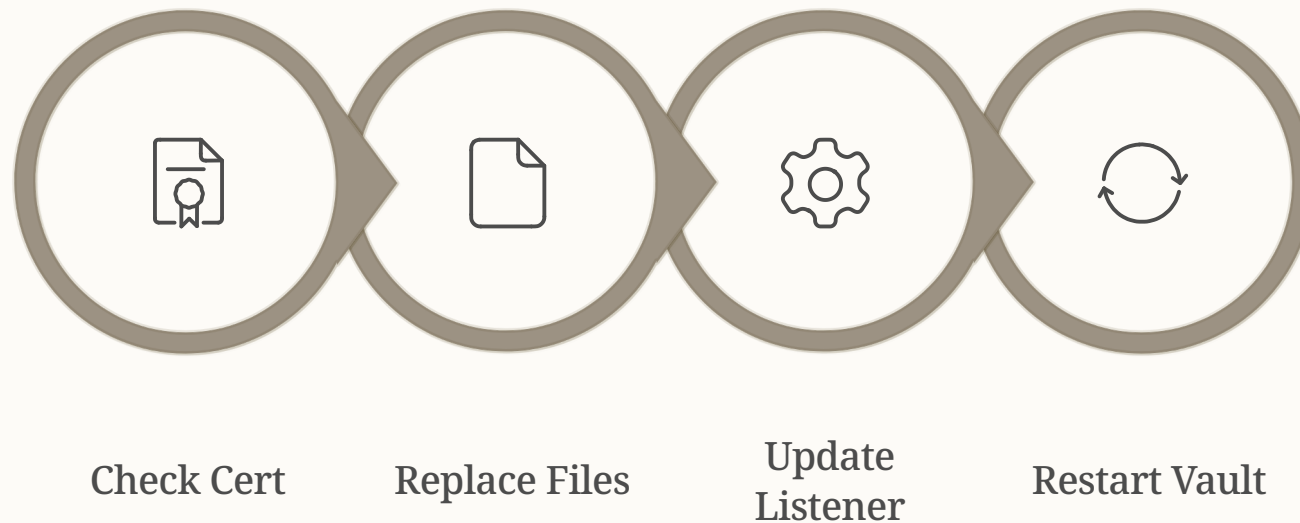
Ensure the managed identity has **Key Unwrap** and **Key Get** permissions on the key vault. Verify `tenant_id`, `client_id`, and `vault_name` values in config.



Manual Unseal

If auto-unseal fails, fall back to manual unseal using `vault operator unseal` with the required threshold of key shares. Confirm quorum is met across nodes.

Resolving TLS Certificate Issues



Common TLS Errors

- `tls: failed to find any PEM data` — wrong file path in config
- `x509: certificate has expired` — renew via your CA or Let's Encrypt
- `tls: private key does not match` — cert/key mismatch, regenerate the pair

Config Reference

```
listener "tcp" {  
  tls_cert_file =  
    "/etc/vault/tls/cert.pem"  
  tls_key_file =  
    "/etc/vault/tls/key.pem"  
}
```

Storage Backend Failures

→ Consul Backend

Ensure the Consul agent is running and healthy on the Vault host. Verify the `address`, `path`, and ACL token in the storage "consul" block. Run `consul members` to confirm cluster health.

→ Integrated Raft Storage

Check that the `raft_path` directory exists, has correct permissions (`vault:vault`), and is not corrupted. Use `vault operator raft list-peers` to inspect cluster membership.

→ etcd Backend

Validate etcd endpoint connectivity with `etcdctl endpoint health`. Confirm TLS certs for etcd client auth are valid and that the Vault service account has read/write access.

Env Var Conflicts & Permission Errors

Environment Variable Conflicts

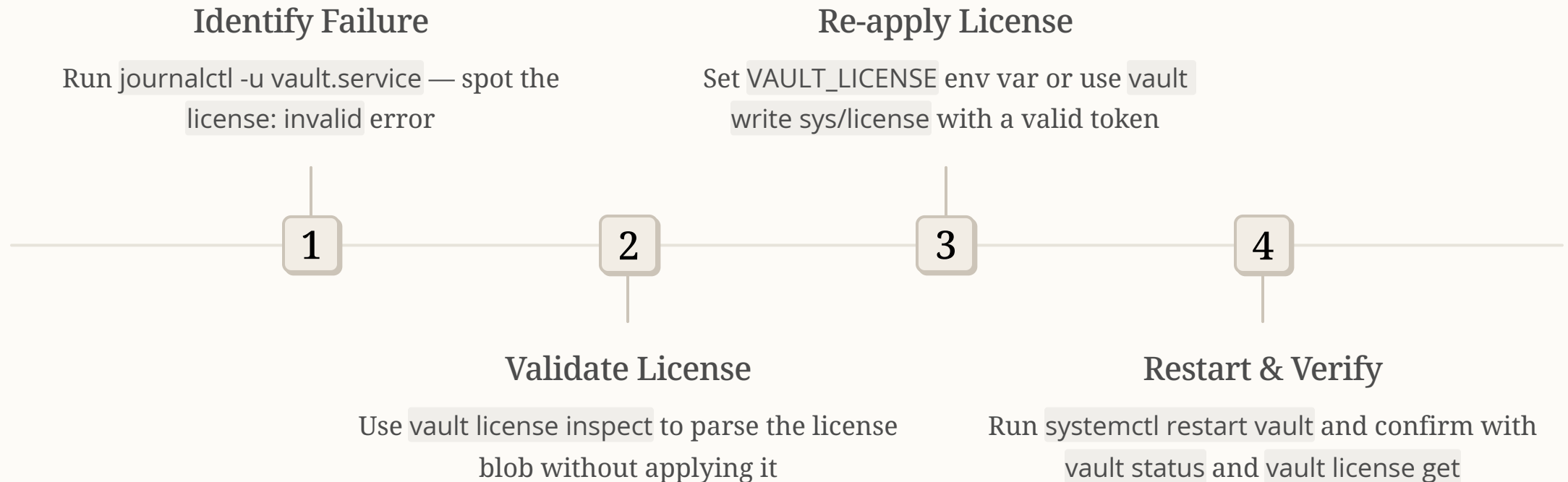
`VAULT_CONFIG_PATH`, `VAULT_LICENSE`, and `VAULT_ADDR` set in shell profiles or systemd unit files can silently override the config file. Always audit `/etc/vault.d/` and the systemd `Environment=` directives together.

File Permission Errors

The Vault process must own its config, TLS, and data directories. A common failure point is post-upgrade permission resets. Fix with:
`chown -R vault:vault /etc/vault.d /opt/vault/data`



Step-by-Step Recovery After a Failed License Update



Chef Automate LDAP Authentication

Chef Automate uses LDAP/Active Directory to authenticate users through its **Identity & Access Management (IAM)** layer. Understanding the flow helps isolate where failures occur.

1

User Login Request

User submits credentials to the Chef Automate UI or API

2

LDAP Bind & Search

Automate binds using `bind_dn` and searches for the user entry

3

Group Resolution

Group memberships are resolved via `member_of` attribute

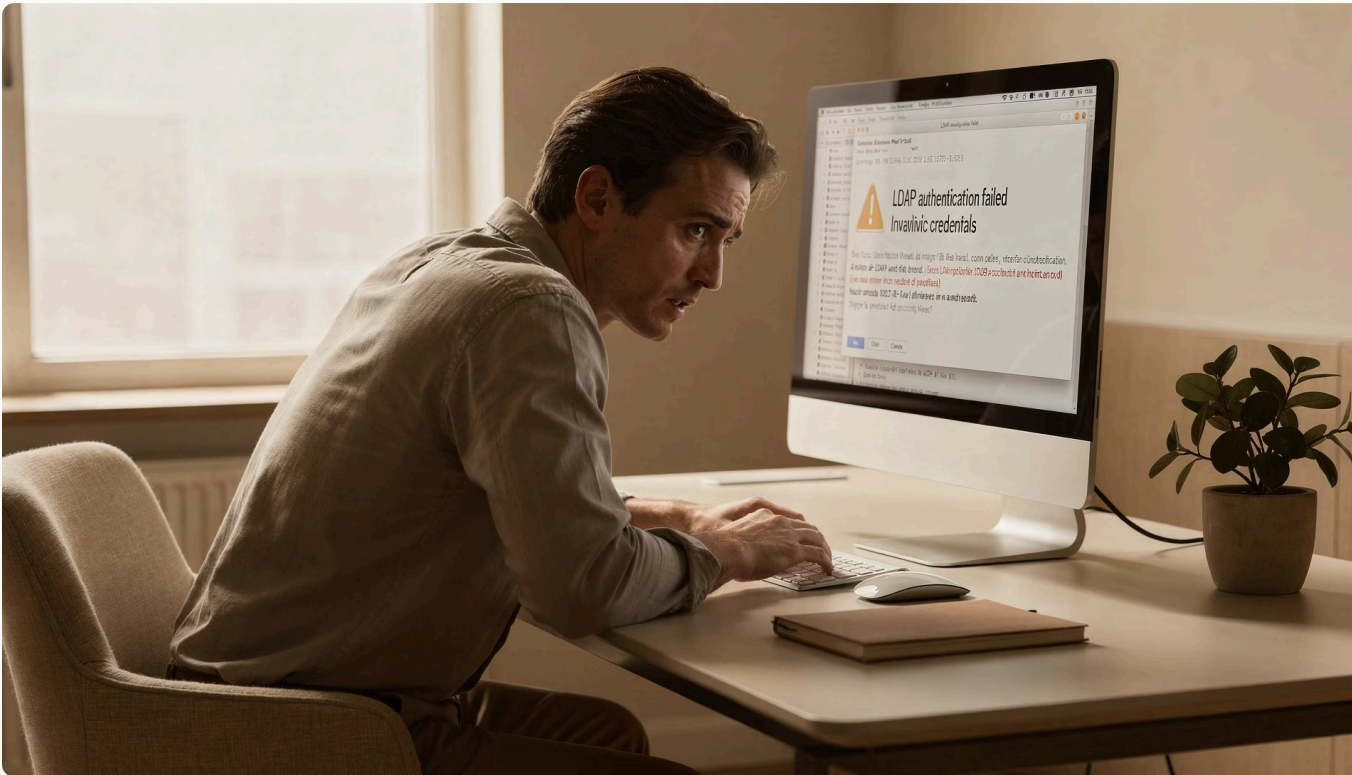
4

IAM Policy Match

Groups are matched to Chef Automate IAM policies for access



The Confusing Scenario: Users in Policy, Can't Sign In



What You See

LDAP users are **correctly listed in Chef Automate IAM policies**, their group memberships are visible, yet every login attempt returns an **authentication failure** or silent redirect to the login page.

Why It's Confusing

The IAM policy layer and the LDAP authentication layer are **separate systems**. A user can be in a policy but still fail authentication if the LDAP connector configuration is misaligned. The policy is never reached if LDAP auth fails first.

Mismatched bind_dn or base_dn

The Problem

If bind_dn uses an account without sufficient permissions, or base_dn doesn't match the correct OU structure, the LDAP search returns **no results** — even when users clearly exist. This manifests as a silent auth failure with no descriptive error shown to the user.

The Fix

In chef-automate.toml, verify:

```
bind_dn = "cn=svc-  
automate,ou=ServiceAccounts,dc=corp,dc=com"  
base_dn = "ou=Users,dc=corp,dc=com"
```

Test the bind account separately using ldapsearch to confirm it can query the target OU with the expected credentials.

Group Attribute Mapping Errors

The Case Sensitivity Trap

Chef Automate's LDAP connector maps group memberships using an attribute name that **must exactly match** what your LDAP schema returns. The most common mistake:

 Wrong

`member_of` (underscored, lowercase)

How to Verify

Run an `ldapsearch` query directly against your directory and inspect the raw attribute names returned:

```
ldapsearch -x -H ldap://your-ldap \  
-D "bind_dn" -w "password" \  
-b "base_dn" "(uid=testuser)" \  
memberOf
```

Copy the exact attribute casing into your `chef-automate.toml` LDAP connector configuration.

 Correct

`memberOf` (camelCase — standard AD attribute)

Token Scope & Policy Attachment in Chef Automate IAM



Policy Assignment Isn't Enough

Adding an LDAP group to a Chef Automate policy grants authorization, but the **authentication token must carry the correct team scope**. If the LDAP team is not properly mapped in Automate's team management, the token is issued without the expected claims.



Verify Team Mapping

Navigate to **Settings** → **Teams** in Chef Automate. Confirm that the LDAP group name (e.g., `CN=chefadmins,OU=Groups,DC=corp,DC=com`) is mapped to the correct Automate team — using the **exact DN format** returned by LDAP.



Re-attach Policies

After correcting team mappings, re-attach all relevant IAM policies to the team. Changes to team membership or policy bindings take effect on the **next login attempt**, not retroactively.

Debugging Checklist: LDAP Auth Failures

Chef Automate Logs

```
$ chef-automate log tail \
  automate-dex
```

The `automate-dex` service handles LDAP authentication. Look for `ldap: bind error`, `no such object`, or `invalid credentials` messages.

LDAP Test Tools

```
$ ldapsearch -x -H ldap://server \
  -D "bind_dn" -w "pass" \
  -b "base_dn" "(uid=user)"
```

Validate connectivity, bind, and search independently from Chef Automate before adjusting config.

✓ Checklist

- Confirm `bind_dn` account is not locked or expired
- Verify `base_dn` matches the correct OU
- Check `memberOf` attribute casing in schema
- Validate team DN mapping in Automate UI
- Re-attach IAM policies after team changes
- Test LDAP connectivity from Automate server host

Key Takeaways & Next Steps

License Management

Keep config and API-applied licenses in sync. Monitor expiry dates and test license changes in non-prod first.

Vault Recovery

Always start with journalctl logs. The five root causes — seal, TLS, storage, env vars, permissions — cover 95% of failures.

Chef Automate LDAP

Auth and authorization are separate. Verify bind_dn, memberOf casing, and team-to-policy mappings independently.

 **Resources:** [HashiCorp Vault License Docs](#) · [Consul License Docs](#) · [Chef Automate LDAP Guide](#)

